CSE 4101: Artificial Intelligence

Lecture Slide 4: Informed (Heuristic) Search Strategies

Prepared By

Mr. Palash Roy

Lecturer

Department of Computer Science & Engineering

University of Dhaka

Where We Are Going

1	**Why informed search?**	Use domain knowledge to search faster than blind (uninformed) search.
2	**Heuristic functions h(n)**	What they are, and different concrete ways to build one.
3	**Greedy best-first search**	Follow whatever looks closest to the goal.
4	**A* search**	Balance cost-so-far and estimated cost-to-go — optimal and efficient.
5	**Admissibility & consistency**	What makes a heuristic trustworthy.
6	**Weighted A*, beam search, IDA*, RBFS, SMA***	Trading optimality or memory for speed.

Informed vs. Uninformed Search

Uninformed Search

- Knows nothing about how far a state is from the goal
- Examples: BFS, DFS, Uniform-Cost Search
- Explores blindly — may expand many useless nodes
- Guaranteed correct, but can be slow

Informed Search

- Use domain knowledge to search faster than blind (uninformed) search.
- Uses a heuristic function h(n) as a hint
- Examples: Greedy Best-First, A*, Weighted A*
- Directs search toward the goal
- Can be dramatically faster — if the heuristic is good

The Heuristic Function h(n)

h(n) = estimated cost of the cheapest path from node n to a goal state

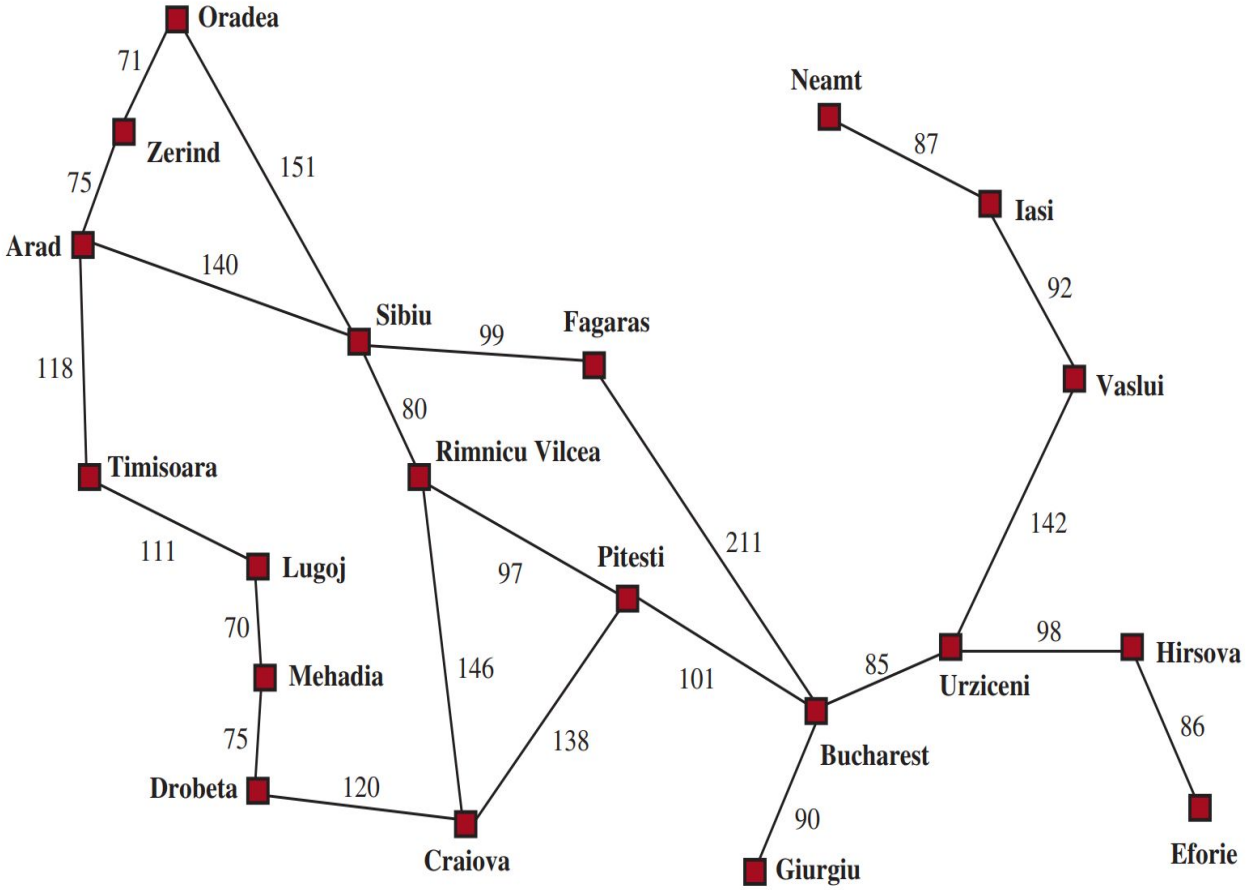
- Computed only from the current state description — **it is a *guess*, not an exact answer**
- Comes from domain-specific knowledge (relaxed versions of the problem, geometry, patterns)
- A perfect heuristic would make search instant; a useless one (h(n) = 0) reduces A* to Uniform-Cost Search

Goal: reach Bucharest

hSLD(n) = straight-line distance from n to Bucharest

hSLD(Arad) = 366 km **hSLD(Sibiu) = 253 km**

hSLD(Fagaras) = 176 km **hSLD(Bucharest) = 0 km**



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

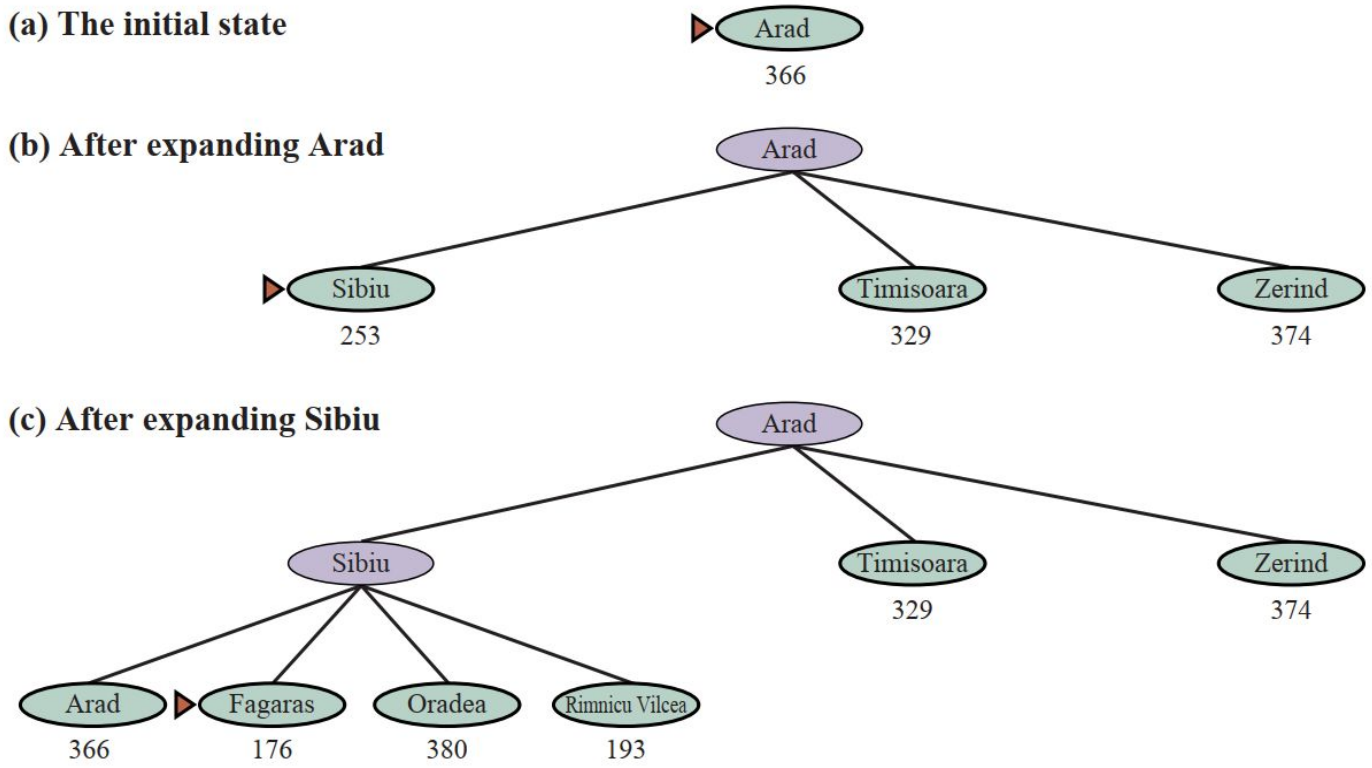
Greedy Best-First Search — The Idea

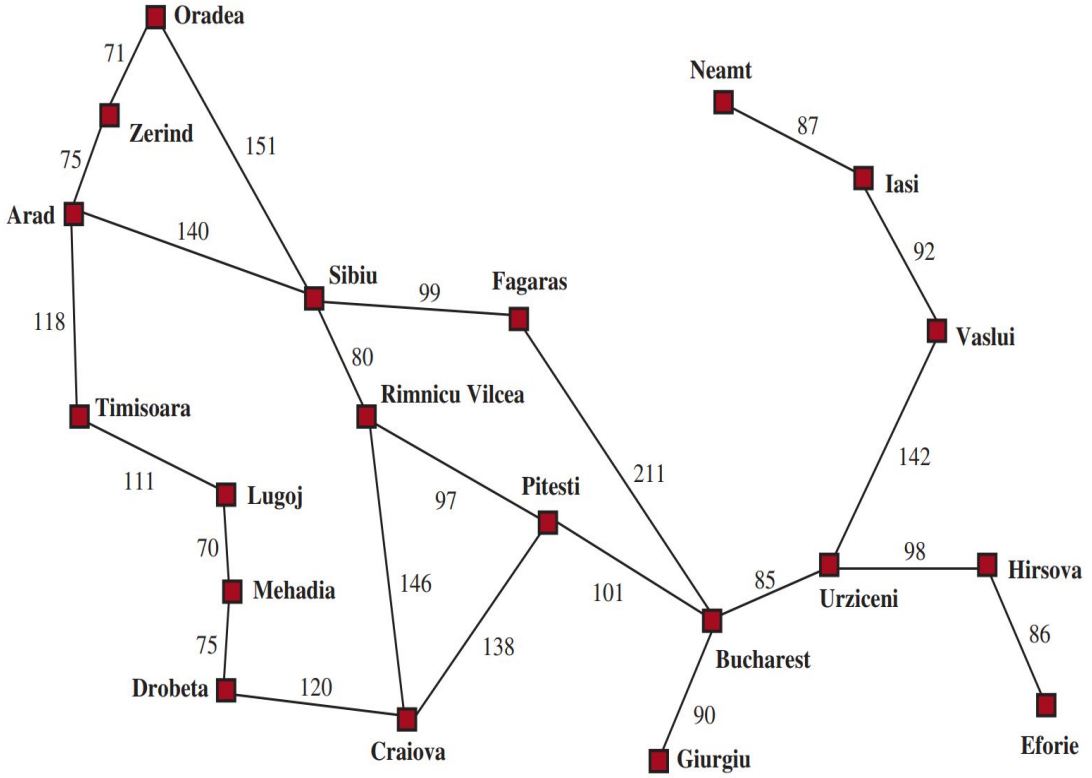
Evaluation function: **f(n) = h(n)**

- Always expand the node that *looks* closest to the goal
- Ignores how much it already cost to get there (g(n) is not used)
- Fast in easy cases — but **can be fooled by a heuristic** that looks good locally yet leads to a long detour
- **Not optimal** — and not even complete on infinite state spaces

Think of it as a hiker who always walks toward the point that looks nearest to camp — even if that means walking straight into a cliff.

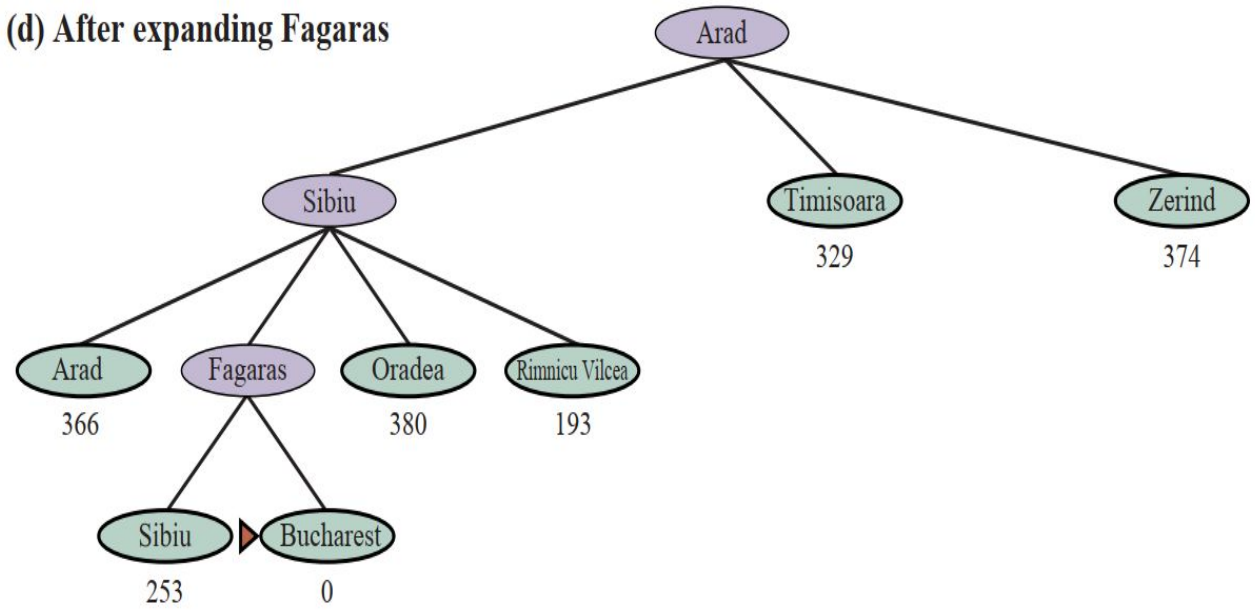
Greedy Best-First — Worked Example: Romania

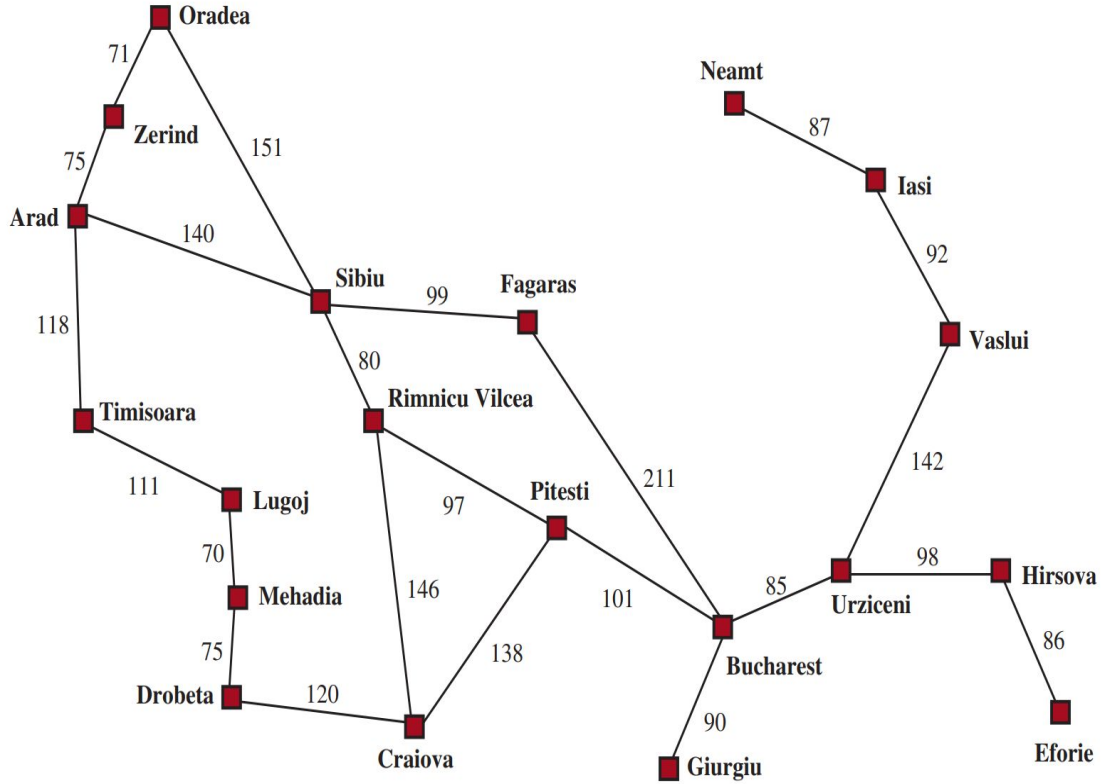




Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Greedy Best-First — Worked Example: Romania





Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Greedy Best-First — Worked Example: Romania

Goal: travel from Arad to Bucharest, using hSLD (straight-line distance to Bucharest)

Step	Node expanded	h(n)	Why chosen
1	Arad	366	Start node
2	Sibiu	253	Lower h than Zerind (374) or Timisoara (329)
3	Fagaras	176	Lowest h among Sibiu's children
4	Bucharest	0	Goal reached!

Result: Path Arad → Sibiu → Fagaras → Bucharest = 140 + 99 + 211 = **450 km**. The optimal path (via Rimnicu Vilcea, Pitesti) is only **418 km** — greedy search missed it by 32 km because it never looks back.

Your Turn to Simulate: Greedy on a Grid

A robot sits on a grid. At every cell, h(n) = Manhattan distance to the goal (shown in each cell below).

The robot always steps into the neighboring cell with the **lowest h-value** — that is greedy best-first search in action.

Start (h=6)

6	5	4	5	6
5	4	3	4	5
4	3	2	3	4
3	2	1	2	3
2	1	0	1	2

Goal (h=0)

Greedy path:

(0,0)→h=6

(1,1)→h=4

(2,2)→h=2

(3,2)→h=1

(4,2)→h=0 ✓

The heuristic pulls the robot straight toward the goal — no backtracking needed here because the grid is open.

Properties of Greedy Best-First Search

Criterion	Result	Why
Complete?	Yes, if finite state space	Otherwise it may loop forever down a bad path
Optimal?	No	It never reconsiders cost already paid
Time	$O(b^m)$, often much better	Depends heavily on quality of heuristic
Space	$O(b^m)$	Keeps every generated node in memory

b = branching factor, m = maximum depth of the search space

A* Search — The Idea

Evaluation function: **f(n) = g(n) + h(n)**

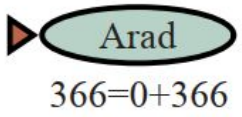
g(n)	h(n)
cost already paid to reach n from the start state	estimated cost still needed to reach the goal from n

f(n) is our best estimate of the **total cost of the cheapest solution through n**.

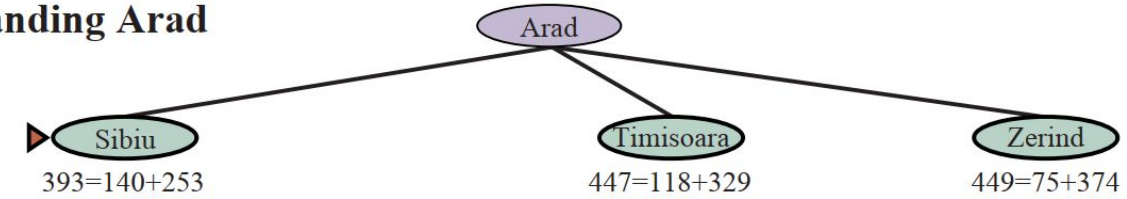
- A* expands the frontier node with the **lowest f(n)** — unlike greedy search, it never forgets the cost already paid
- It is complete and, with the right heuristic, optimal — the gold standard of informed search

A* Search — Worked Example: Romania

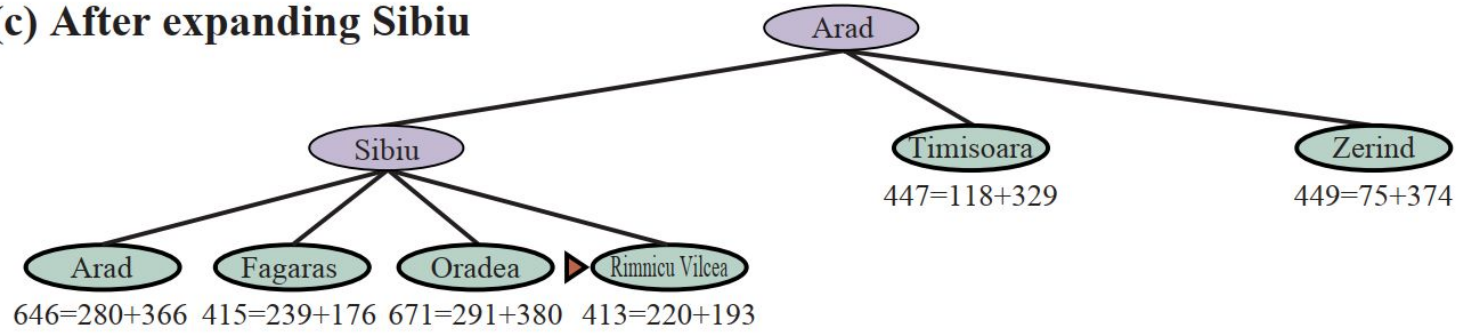
(a) The initial state

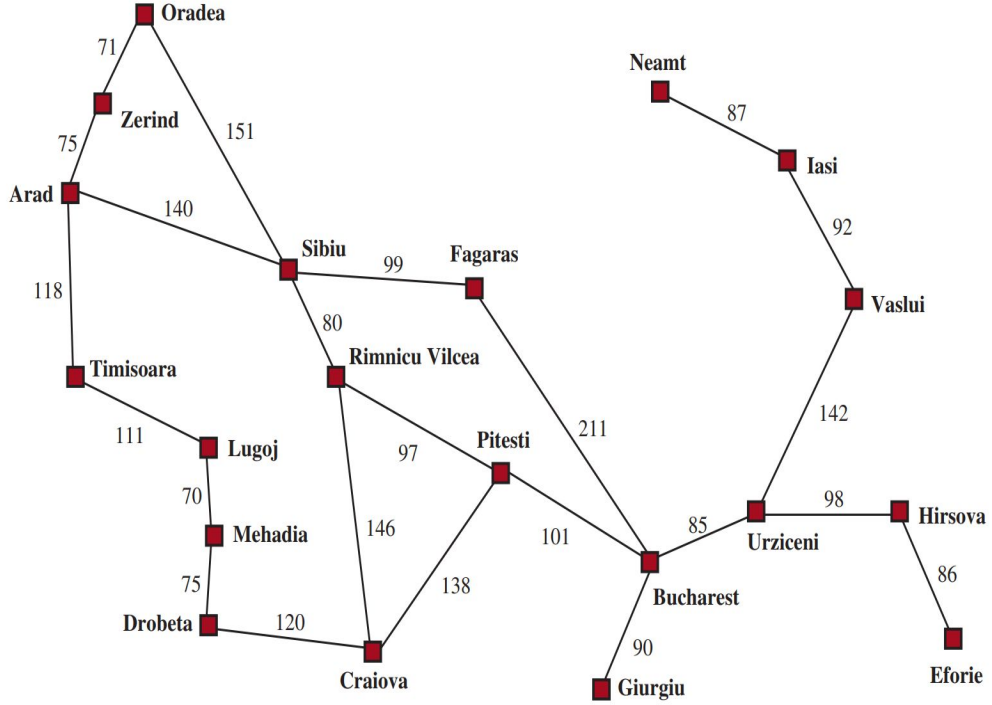


(b) After expanding Arad



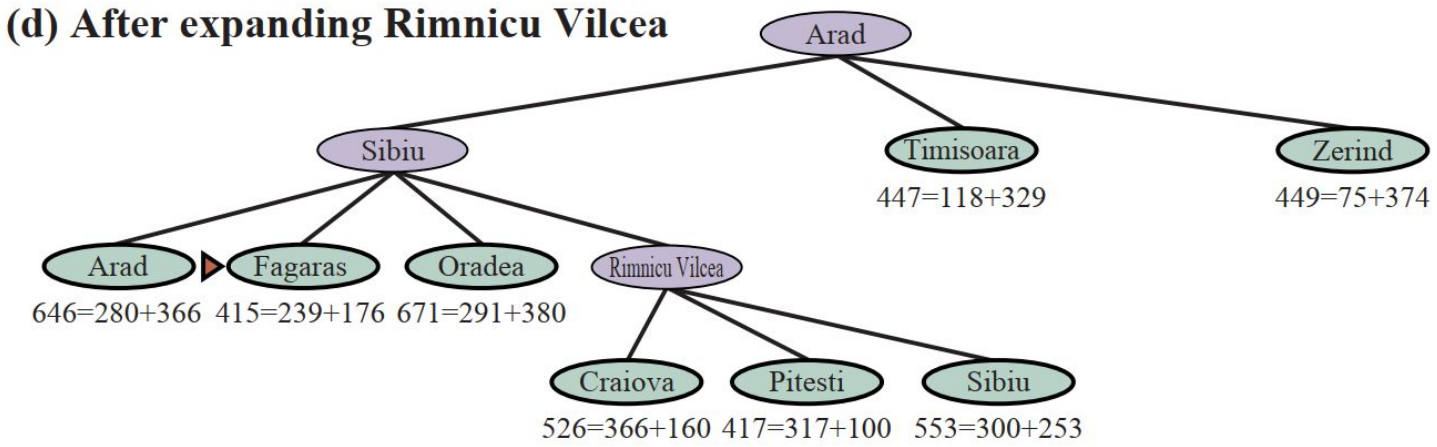
(c) After expanding Sibiu

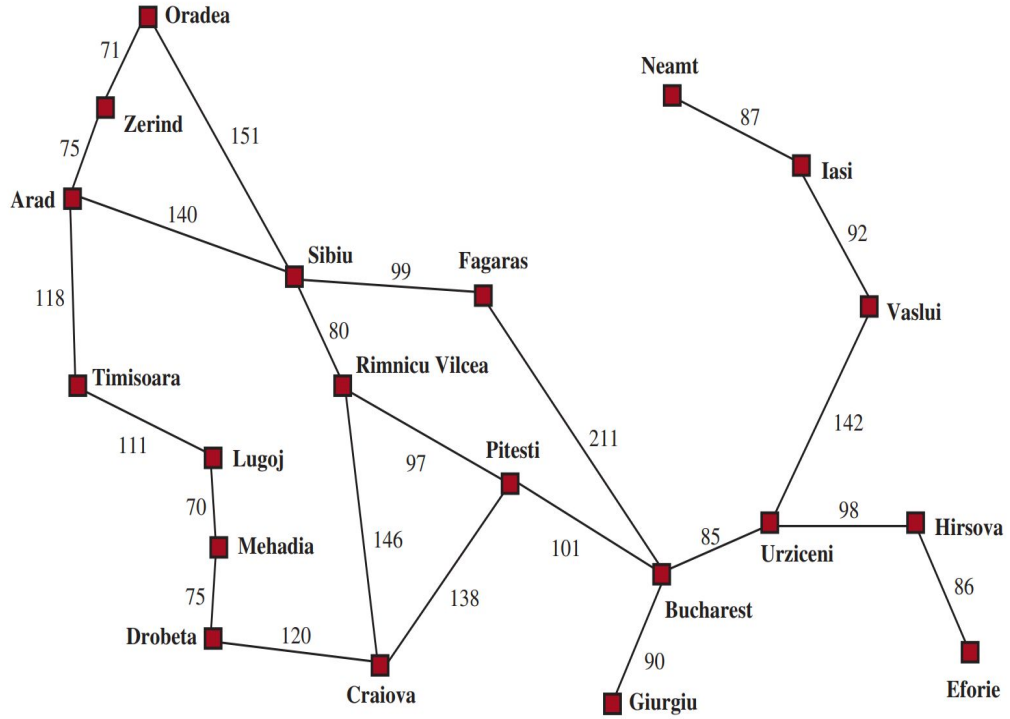


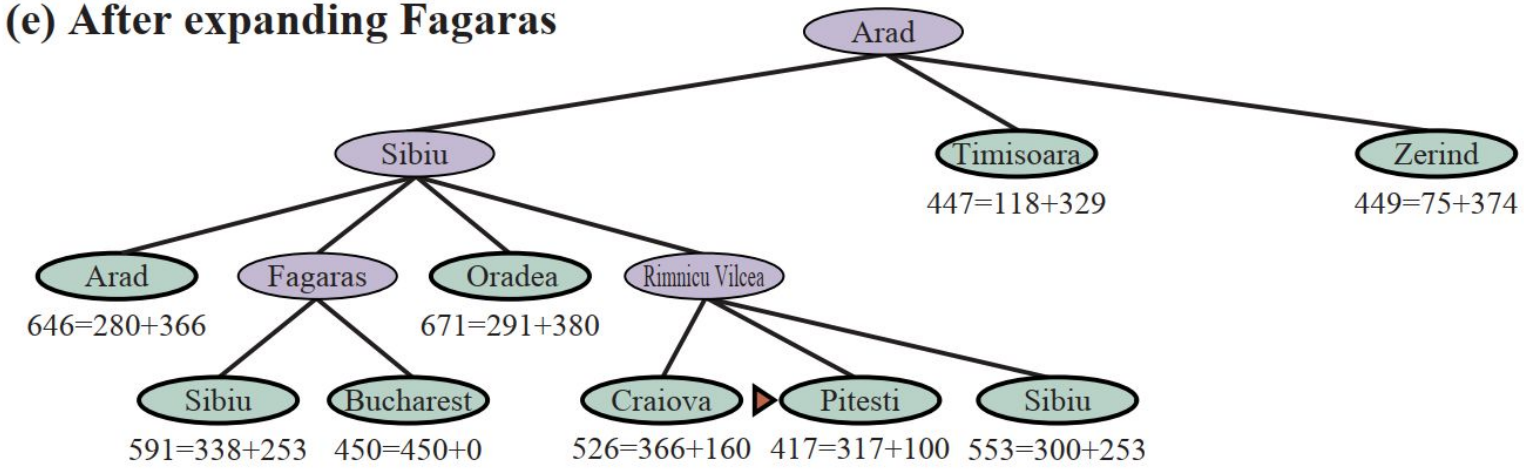


Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

A* Search — Worked Example: Romania

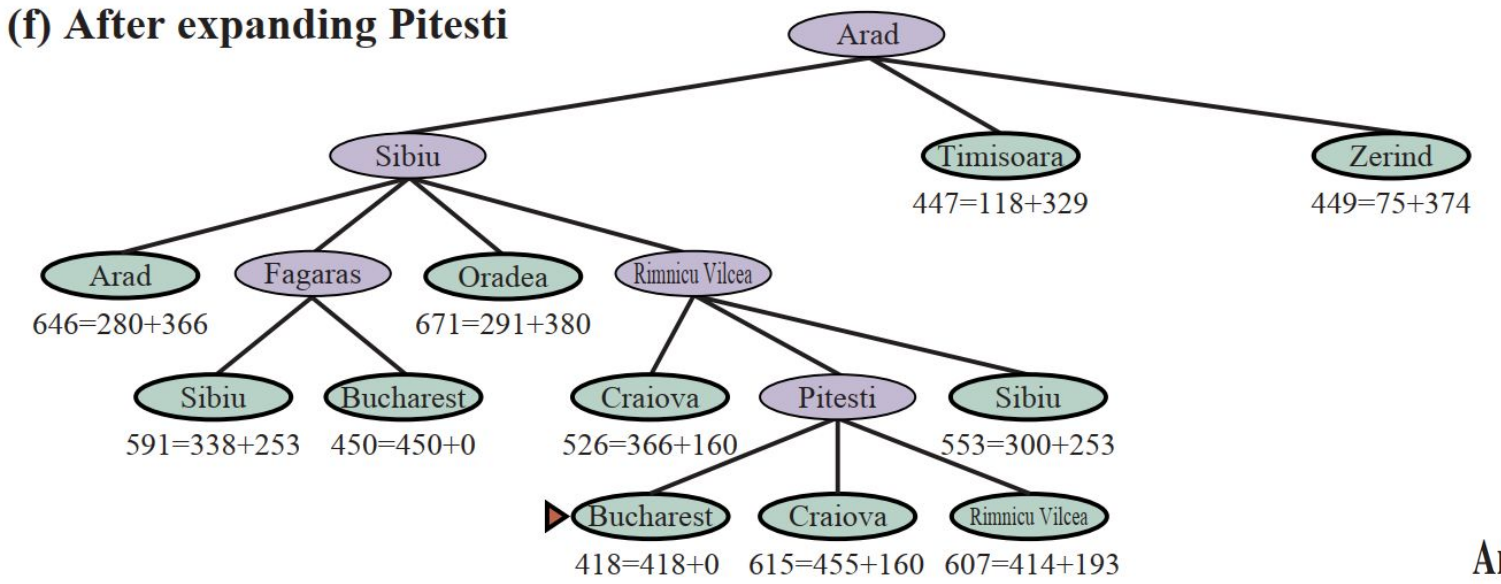


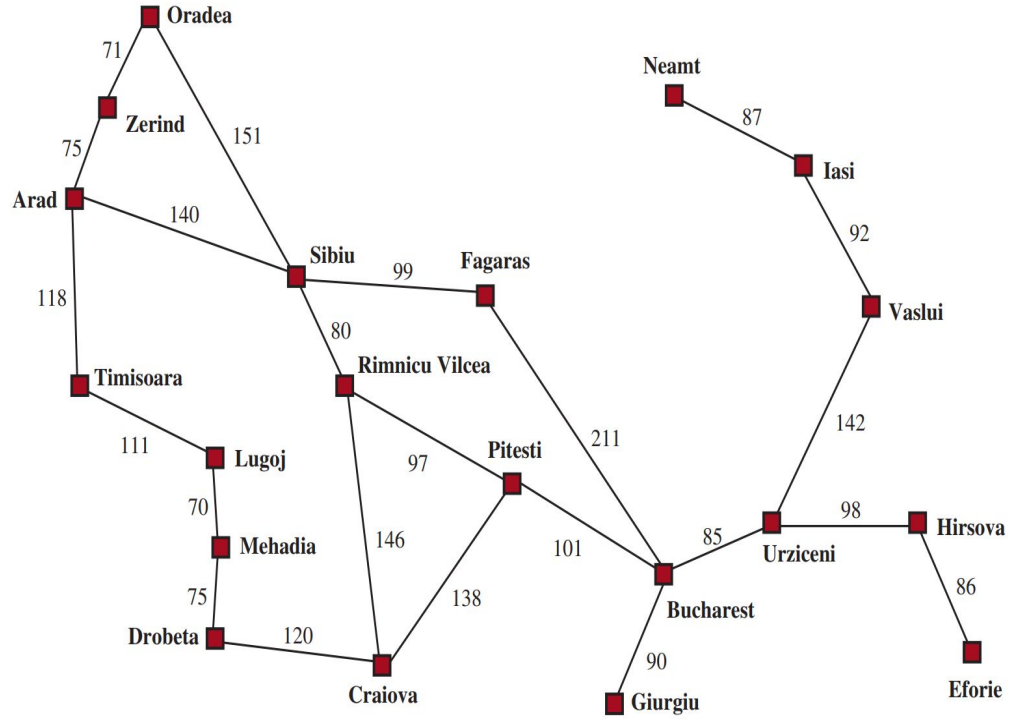




Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

A* Search — Worked Example: Romania





Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

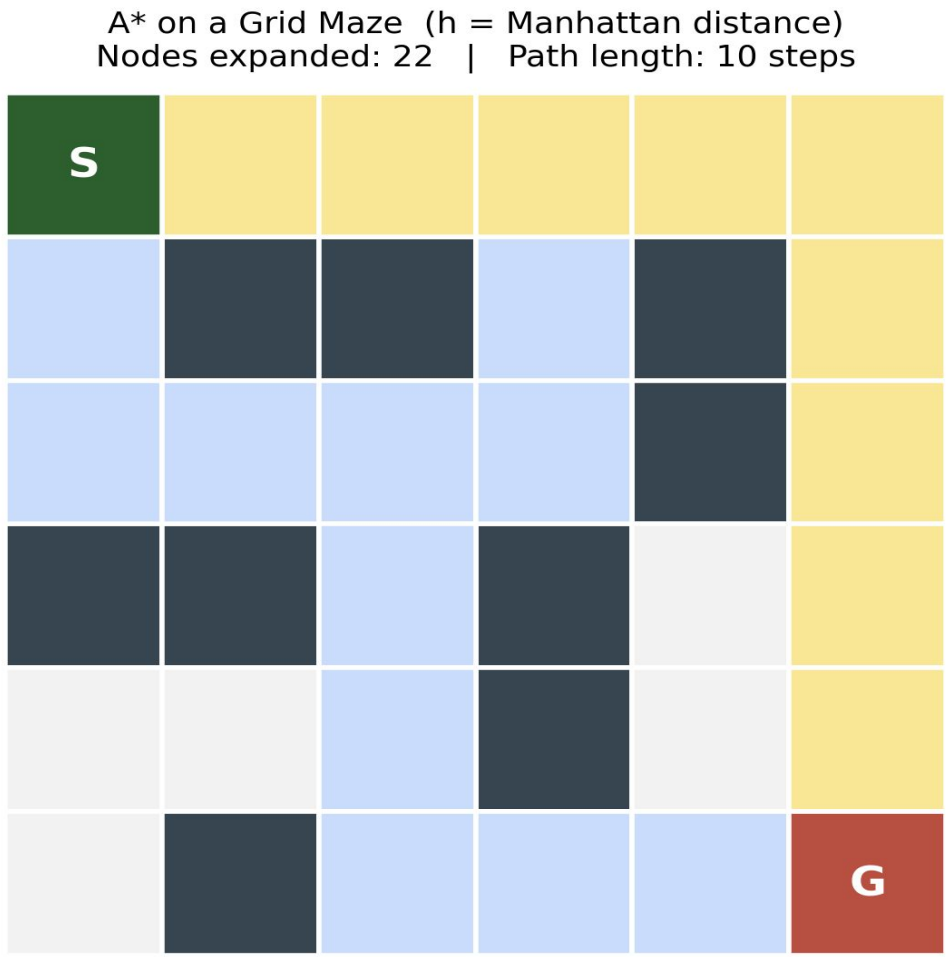
A* — Worked Example: Romania

Same problem as before, but now f(n) = g(n) + h(n). Watch how A* refuses to settle early.

Step	Frontier lowest f	g	h	f = g+h
1	Arad	0	366	366
2	Sibiu	140	253	393
3	Rimnicu Vilcea	220	193	413
4	Pitesti	317	100	417
5	Bucharest (via Pitesti)	418	0	418 ✓ optimal

Key moment: Bucharest first appears on the frontier at f=450 (via Fagaras), but A* does *not* stop there, because Pitesti still has a lower f=417. Only once no frontier node beats 418 does A* accept Bucharest — guaranteeing the optimal path.

Simulating A* on a Grid Maze



Setup

- 6×6 grid, dark cells are walls
- Start (S) top-left, Goal (G) bottom-right
- h(n) = Manhattan distance to G
- Each step costs 1

What happened

- Blue cells: expanded by A* but not on final path
- Yellow cells: the optimal path found
- A* explored only 22 of 36 cells — far fewer than BFS would need
- The path shown is guaranteed shortest, because Manhattan distance never overestimates true cost here

Admissible Heuristics

A heuristic h(n) is admissible if: $0 \leq h(n) \leq h^*(n)$

where h(n) is the true cost of the cheapest path from n to the goal.*

- An admissible heuristic **never overestimates** — it is always **optimistic**
- Straight-line distance is admissible for route-finding: the direct line can never be longer than the real road distance
- Guarantee: if h(n) is admissible, A* tree search is cost-optimal (proof by contradiction on the next slide)

Own example: in the 8-puzzle, the number of misplaced tiles is admissible — each misplaced tile needs at least one move, so the count can never exceed the true number of moves needed.

Why Admissibility Guarantees Optimality

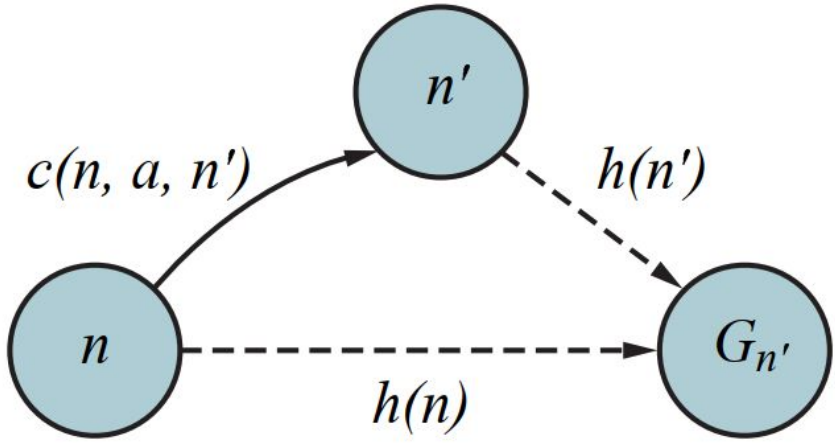
Proof sketch (by contradiction): suppose A* returns a path of cost C, but the optimal cost is C* < C.

1. Some node n on the optimal path must still be unexpanded (else we'd have returned the optimal path).
2. $f(n) > C^*$ — otherwise A* would have expanded n first.
3. $f(n) = g(n) + h(n) = g^*(n) + h(n)$ since n is on the optimal path, $g(n) = g^*(n)$.
4. Because h is admissible, $h(n) \leq h^*(n)$, so $f(n) \leq g^*(n) + h^*(n) = C^*$.
5. But that contradicts step 2 ($f(n) > C^*$). Contradiction ⇒ A* cannot return a suboptimal path.

A Stronger Property: Consistency

h(n) is consistent if, for every successor n′ of n via action a: **$h(n) \leq c(n,a,n') + h(n')$**

- This is a form of the **triangle inequality**: one side of a triangle can't be longer than the sum of the other two
- Straight-line distance is consistent, not just admissible
- Every consistent heuristic is admissible (converse is not always true)
- With a consistent heuristic, the f-values along any path are monotonically non-decreasing — so the first time A* reaches a state, it is already on an optimal path to that state



A* Is Optimally Efficient

Pruning: eliminating possibilities from consideration without ever examining them.

In the Romania example, Timisoara (f=447) and Zerind (f=449) are children of the root and would be among the very first nodes expanded by BFS or uniform-cost search — but A* **never expands them at all**, because the f=418 solution is found first.

With a consistent heuristic, A* is **optimally efficient**: no other algorithm that extends search paths from the start and uses the same heuristic information can guarantee expanding fewer nodes.

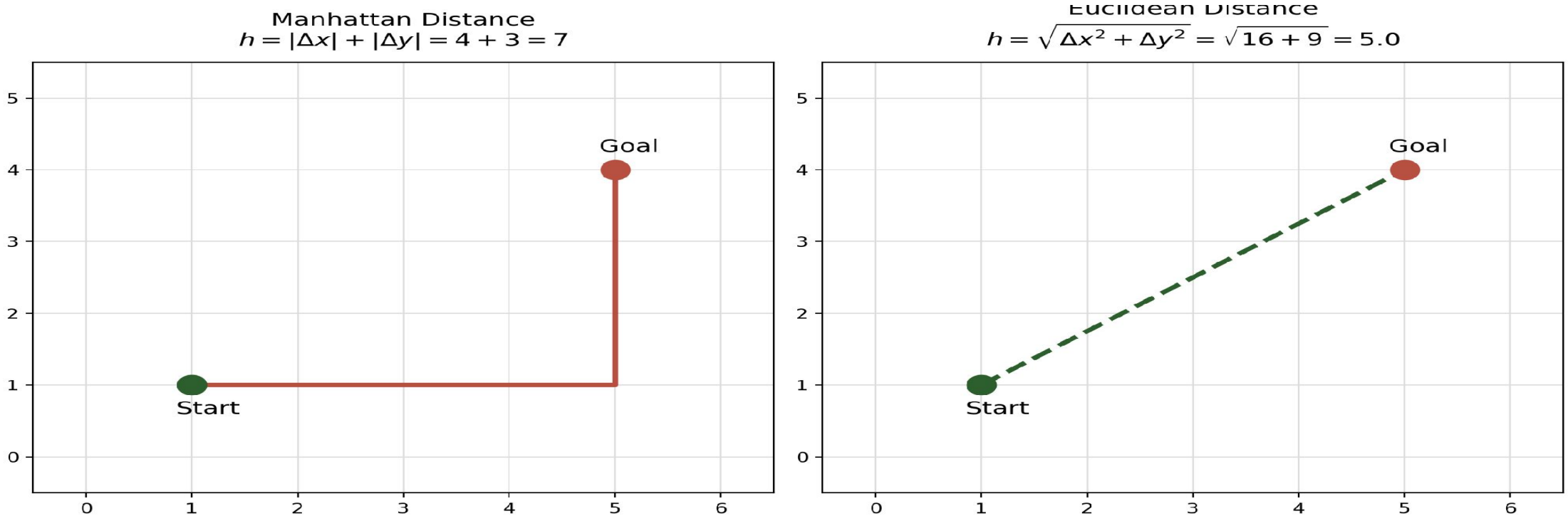
Caveat: this doesn't mean A* is cheap — for some problems (e.g. certain vacuum-world variants) the number of nodes on the optimal-cost frontier is itself exponential in the solution length.

Types of Heuristic Functions

Distance-based	Relaxed-problem	Pattern databases	Landmark / weighted
Manhattan / Euclidean distance	Misplaced tiles, MST of unconnected cities	Precomputed exact costs for sub-problems	Detour index, learned weights
Grid worlds, robotics, route-finding	*Remove a constraint, solve the easier problem*	*8-puzzle corner patterns, Rubik's Cube*	*Real road networks, satisficing search*

We will look at concrete, worked examples of each of the first two families on the following slides.

Heuristic Type 1: Distance-Based

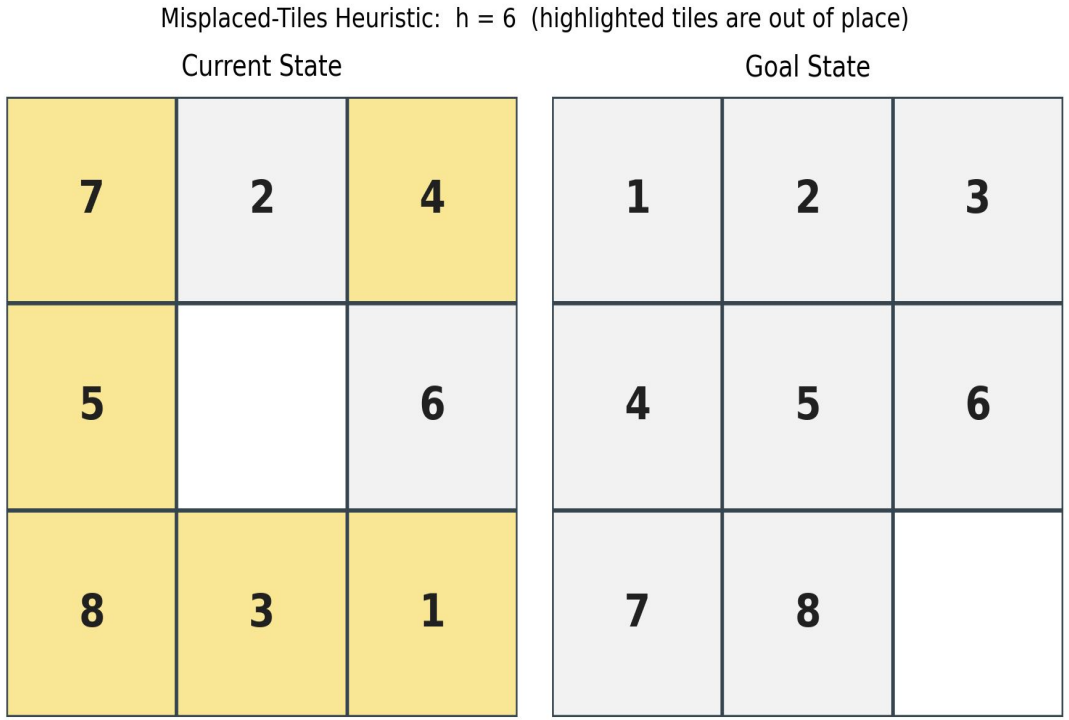


Manhattan distance — admissible when movement is restricted to grid steps (no diagonals). Common in warehouse robots, tile puzzles.

Euclidean distance — admissible when movement can be in any direction. Common in continuous-space robotics and map routing.

Heuristic Type 2: Relaxed-Problem Heuristics

Original example — 8-puzzle



Misplaced-tiles heuristic

Relax the rule that tiles slide one at a time — imagine any tile can jump directly to its goal square.

Then h(n) = number of tiles not in their goal position (ignoring the blank).

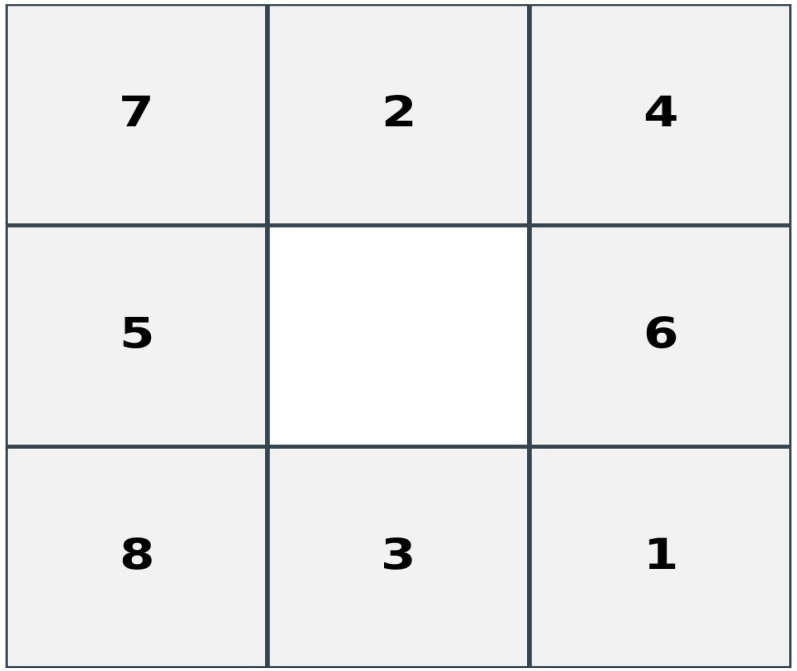
h(n) = 6 for the state shown (6 of 8 numbered tiles are out of place).

This is admissible: every misplaced tile needs ≥1 real move, so it never overestimates.

A Better Heuristic: Manhattan Distance

Original example — same puzzle

Manhattan-Distance Heuristic for the 8-Puzzle



Sum, over every tile, of the number of grid moves (up/down/left/right) it needs to reach its goal square.

For the same state:

h(Misplaced) = 6 **h(Manhattan) = 14**

Both are admissible, but Manhattan distance is **more informed** (dominates): it is always ≥ the misplaced-tiles count, while still never exceeding the true cost. A more informed admissible heuristic lets A* expand fewer nodes.

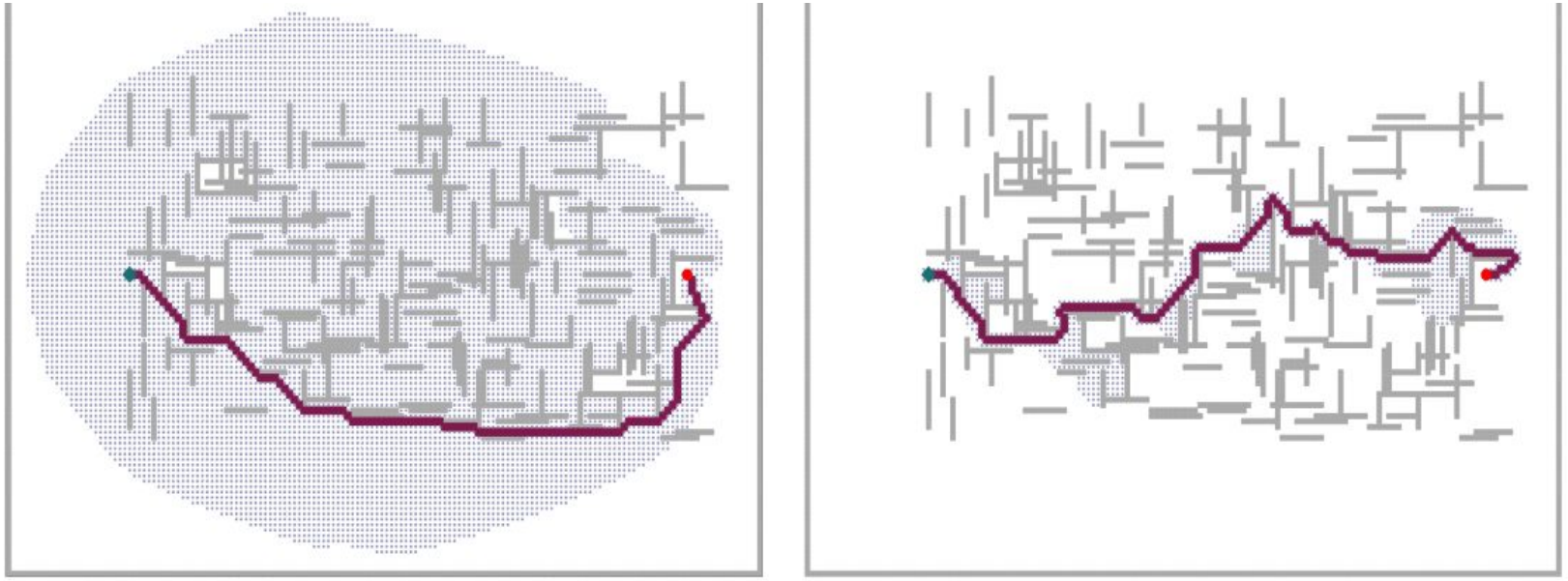
Satisficing Search: Weighted A*

$f(n) = g(n) + W \times h(n)$, for some $W > 1$

- Weighting h more heavily biases search toward states that look closer to the goal — a controlled dose of 'greediness'
- If the optimal cost is C*, weighted A* returns a solution between C* and W×C* — in practice usually much closer to C*
- A useful family view:

Algorithm	Evaluation function	Weight
Uniform-cost search	g(n)	W = 0
A* search	g(n) + h(n)	W = 1
Weighted A*	g(n) + W·h(n)	$1 < W < \infty$
Greedy best-first	h(n)	$W = \infty$

Weighted A* in Practice

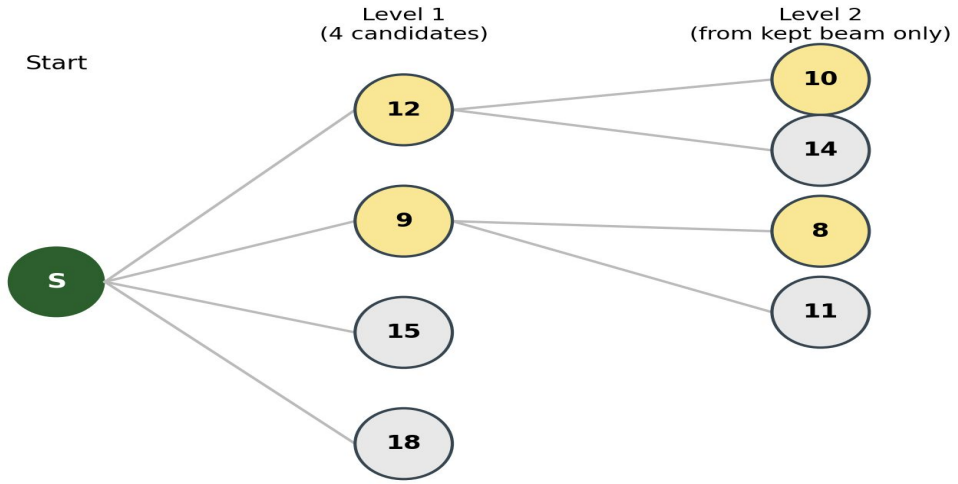


(a) Plain A*: explores a large fan of states to guarantee the optimal path.

(b) Weighted A* (W=2): explores 7× fewer states and finds a path only 5% costlier — a strong trade when speed matters more than perfection.

Memory-Bounded Search: Beam Search

A* keeps every generated node in memory — for large problems, it runs out. Memory-bounded methods trade completeness or optimality for a bounded memory footprint.



Beam search

Keep only the k best f-scored nodes at each level; discard the rest — even if one of them was actually on the optimal path.

Fast, small memory, but incomplete and can miss the optimal (or even any) solution if k is too small.

Iterative-Deepening A* (IDA*)

Idea: combine A*'s guidance with iterative deepening's tiny memory footprint. Instead of cutting off by depth, IDA* cuts off by f-cost.

1. Set the f-cost limit to f(start) = h(start).
2. Depth-first search, but never expand a node whose f(n) exceeds the current limit.
3. If no solution is found, set the new limit to the smallest f-value that exceeded the old limit.
4. Repeat — each iteration exhaustively searches one more f-contour.

Memory used: O(depth) — only a single number (the current f-limit) is kept between iterations. Cost: some states get re-expanded every iteration.

Recursive Best-First Search (RBFS)

Mimics best-first search using only linear space via an f_limit — the best alternative path known from any ancestor.

```
function RBFS(problem, node, f_limit):
  if node is a goal: return node
  successors ← EXPAND(node); if empty: return failure, ∞
  for each s in successors: s.f ← max(s.g + h(s), node.f)
  loop:
     best ← successor with lowest f
     if best.f > f_limit: return failure, best.f
     alt ← second-lowest f among successors
     result, best.f ← RBFS(problem, best, min(f_limit, alt))
     if result ≠ failure: return result
```

On the Romania problem, RBFS follows Rimnicu Vilcea, backtracks to try Fagaras, then returns once Fagaras's best leaf is beaten — mirroring A's order using only O(depth) memory.*

Simplified Memory-Bounded A* (SMA*)

- Behaves exactly like A* until memory fills up
- When full, drops the worst leaf node (highest f) to make room for a new one
- Backs up the forgotten leaf's f-value to its parent — so the parent still 'remembers' how promising that branch was
- If everything else looks worse than the forgotten branch, SMA* regenerates it later

Complete if a reachable solution exists within the memory budget; **optimal** if an optimal solution is reachable. On very hard problems it can *thrash* — repeatedly regenerating the same forgotten nodes, similar to page-thrashing in an operating system — trading time for the memory it saved.

Bidirectional Heuristic Search

Run two simultaneous heuristic searches — forward from the start, backward from the goal — hoping they meet in the middle.

fF(n) = gF(n) + hF(n) for the forward search

fB(n) = gB(n) + hB(n) for the backward search

A practical evaluation function that mimics the theoretical lower bound is:

f2(n) = max(2·g(n), g(n) + h(n))

The two frontiers 'meet in the middle' when neither has any node with g(n) > C*/2.

Summary: Comparing the Algorithms

Algorithm	Complete?	Optimal?	Memory
Greedy Best-First	Yes (finite space)	No	$O(b^m)$
A*	Yes	Yes (admissible h)	$O(b^m)$
Weighted A*	Yes	Bounded ($\leq W \cdot C^*$)	$O(b^m)$
Beam Search	No	No	O(k)
IDA*	Yes	Yes (admissible h)	O(depth)
RBFS	Yes	Yes (admissible h)	O(depth)
SMA*	Yes (if fits)	Yes (if fits)	Bounded (fixed)

Key Takeaways

- A heuristic h(n) injects domain knowledge into search — the better the estimate, the fewer nodes get explored.
- Greedy best-first search is fast but can be led astray; A* fixes this by never forgetting cost already paid: f(n) = g(n) + h(n).
- Admissibility (never overestimate) guarantees A* finds the optimal solution; consistency is a stronger, very convenient guarantee.
- There is no single 'best' heuristic — Manhattan distance, misplaced tiles, and pattern databases each trade accuracy for computation cost.
- When memory or time is tight, weighted A*, beam search, IDA*, RBFS, and SMA* let you trade away optimality or completeness deliberately.

Next: Section 3.6 — Heuristic Functions

Where do heuristics actually come from?

- Designing heuristics for the 8-puzzle in more depth
- Relaxed problems and how they generate admissible heuristics automatically
- Pattern databases and learning heuristics from experience
- Combining multiple heuristics with h(n) = max(h1(n), h2(n), ..., hm(n))

Questions?